

Task 2: Advanced Encryption Standard

Chair of Dependable Nano Computing, Karlsruhe Institute of Technology

Authors: Jonas Krautter, Simon Pankner, Dennis Gnad, Mehdi Tahoori

June 4, 2026

1 Introduction and Supplementary Information

The Advanced Encryption Standard (AES) was introduced by the US National Institute of Standards and Technology (NIST) in 2000 as a replacement for the formerly used Data Encryption Standard (DES), which was considered insecure due to the short key length and increasing computational power. Originally, the algorithm was named Rijndael after its developers Joan Daemen and Vincent Rijmen, who submitted it to a competition which would determine the most suitable DES successor.

AES is a symmetric block cipher, meaning it operates on data blocks of 128 bit length and uses the same key for encryption and decryption. The encryption function $C = Enc(K, M)$ takes a 128 bit input plaintext together with the key, which can have a length of 128, 192 or 256 bit, and computes a 128 bit output ciphertext. If we want to encrypt longer messages, the data needs to be segmented and padded before submitting it to one of the various encryption modes, such as Electronic Codebook (ECB), Cipherblock Chaining (CBC), Cipher Feedback (CFB) or Output Feedback (OFB).

For this task, we focus on the block encryption only, without any encryption mode. You will implement a 128 bit key length AES block cipher module in hardware, that receives plaintexts through the UART interface and sends the respective ciphertext back afterwards. Therefore, you need a detailed understanding of the algorithm, which will also help you in later tasks on attacking the AES cipher. The most important reference document is the original NIST publication NIST.FIPS.197 [1], which is available online and has very helpful example cipher vectors for debugging in the appendix. You can focus on this document for most implementation needs, but you are also encouraged to look for further documentation of the AES online. Since it is a very well known and widely used algorithm, there is an abundance of documentation and examples available.

1.1 Examination of this and following Tasks

As with the previous task, after the task is finished, please submit the files in ILIAS and present the task to us as previously with the PUF task. In this task, the focus is more on the hardware implementation.

You will have time until **Wednesday, 24th June 2026**

1.2 Questions / Help

This task is more complex than the previous one and thus also allows for more freedom. It means that certain concepts you follow might not work, for example because the FPGA might be too small. Please contact us before you are completely stuck, or if you cannot find out what your problem is.

For details on AES, the NIST document [1] (the particular version we linked) can be very helpful, as we describe below. Please take a look into it, it might seem daunting at the start, but it is actually a very helpful document that even guides you through all stages of AES step by step, including example values and which intermediate values to expect.

2 Task Details

2.1 Optional Debug Preparation (recommended!)

For this task we introduce a simulation environment that can help you develop the task significantly faster than testing it on the FPGA each time. For the simulation environment to work you need to make sure to use oss-cad-suite version **2024-11-30** or newer. Please make sure you have sourced the respective environment file like:

```
source <path-to-your-20241030>/oss-cad-suite/environment
```

Then, on the same shell, extend the python environment inside that oss-cad-suite with an AES module, by running:

```
pip3 install aes
```

Now, you can run:

```
make sim
```

to launch the simulation, which performs test on the correctness of your submodules. This will of course fail at the beginning, as your design is not functional, but you can follow up by checking the waveform with:

```
make waveform
```

There, you can look at the signals of your entire design and how they change over time. Please be aware, that this is from the simulation / test environment. It is important to check the timestamps from your `make sim` output to know at which time which part of the overall design component is tested (in nanoseconds at the beginning of the line, for instance starting with “0.00ns INFO cocotb”)

We added the basic clock signals, but you need to add more. For that you can navigate through the hierarchy of your design on the left side. By clicking on the signals, they will be added to waveform on the right side. **Hint 1:** Use Ctrl+Mousewheel to zoom in and out. **Hint 2:** Be aware that not always all signals make sense, depending on which test is run. The UART module is also shown but not really used, so you can ignore that.

Does everything work as expected? Please let us know if something seems wrong with the simulation and waveform debugging as we have just recently added it.

2.2 Task Description

We provide the completed outer framework of the AES hardware encryption module as well as a Makefile to compile the project. You should not have to edit *top_level.v*, *aes_module.v* or *uart.v* **at all**, but are of course allowed to do so, if your implementation requires it. The outer

framework handles the communication with the PC and the basic state machine. It waits until 16 bytes of plaintext data have been received, then activates the encryption and returns the ciphertext through UART after the encryption has finished. Your task is to implement the four main AES operations SubBytes, MixColumns, ShiftRows, AddRoundKey as well as the key expansion (key schedule) part.

We provide some small helper functions, which you may find useful for implementing the AES operations. We provide a simple AES sbox lookup table in *sbox.v*, a helper function for multiplication in $GF(2^8)$ in *xtime.v* as well as the round constants for the key scheduling in *rcon.v*.

The generic interface of each AES operation module is provided but the internal implementation of the submodules is completely up to you. You may use any of the helper functions and can add arbitrary additional modules as well. The makefile automatically includes all Verilog files in the source directory in the compilation. If you wish to change that behaviour, please adapt line 2 of the makefile accordingly. Furthermore, you might not want to implement some operation sequentially, in which case you can simply ignore the *clk* or *rst* signals. The AddRoundKey operation should be implemented in *aes.v* itself (see the TODO in the statemachine there). To summarize: The way you implement the AES operations is up to you, we only provide a helpful skeleton and of course your overall design is restricted by the size of our iCE40-HX8K FPGA.

Please note that the secret AES key is hard-coded into *aes_module.v* as the example key from NIST.FIPS.197. If you wish to evaluate a different key, you must change the file there accordingly.

We provide a python script *check_aes.py* to quickly verify the correct function of your finished AES design.

3 Further Development and Debugging Help

For debugging and development on the FPGA itself, it can be helpful to jump in the state machine (**always**-block) of *aes.v* to the last state (**DONE**) to get the result of an intermediate value to the PC through UART, without going through the other states.

For example, in the beginning, you implement the **SUB_BYTES** state. From that state you can instead of going to the next state, go to the **DONE** state, and you will be able to check the result on the PC with the *check_aes.py* script, if it matches what you expect.

To get some helpful examples what values to try (with known-good results of the intermediate states), the NIST.FIPS.197 document has a very helpful appendix. **You just have to first understand the 4x4 matrix style of writing the aes_state register, which is essential to solve the whole task.**

– **Important Hint** – Regarding potential size limitations: Instantiating a certain module multiple times will usually create that respective hardware structurally (i.e. instantiating the sbox five times, will use FPGA resources for five of them). If you want to perform operations sequentially, you have to schedule it yourself using an **always**-block sensitive to *clk*, and for example by a counting mechanism or state machine. If you are close to filling the FPGA to its limit, you might get seemingly unrelated errors in the mapping process, for instance starting like: “**ERROR: Failed to expand region [...]**” (but there can be others!). Please check how much resources are used with **make report** in case you are unsure.

4 Extra Tasks

We consider some bonus (of not any value as of this writing), if you do the following tasks:

- Try to run your design at different (higher) clock rates and find out the maximum clock speed that still runs fine according to timing analysis. To actually run it, you need to instantiate a PLL with *icepll*. An example file for that is included *example_bonus_task_clkgen48.v*. Please note down the achievable speeds that you could get in your timing report and when using the PLL.
- After checking the achievable speed of your design, you might want to try optimizing it to increase that speed by looking at your critical paths (the *icetime* tool can report it). There is a tradeoff between requiring more clock cycles for an overall operation and being able to increase the frequency to reduce the natural time needed of each clock cycle. Getting into detail on that is usually used in pipelined designs to increase the overall throughput.
- Optimize your design in size, i.e. reduce the amount of needed FPGA resources to a minimum.

References

- [1] National Institute of Standards and Technology (NIST). (2001) Announcing the advanced encryption standard (AES). [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf>